

CMPE 260 Project - Abstract Syntax Tree

Mustafa Şahin - 2023400162
Azra Zülal Muhcu - 2024400372

June 2026

The following table documents the Abstract Syntax Tree nodes used in the interpreter.

Node Type	Fields (Children)	Description
Program	block: Block	The root node of the AST representing the entire program.
Block	statements: List of Statements final_expr: Expression (optional)	A sequence of semicolon-terminated statements, optionally followed by a final expression that acts as the block's return value.
IntLit	value: Integer	An integer literal (e.g., 42).
BoolLit	value: Boolean	A boolean literal (true or false).
VarExpr	name: String	An identifier expression used for variable or function name lookups.
BinOp	op: String left: Expression right: Expression	A binary operation. The operator can be arithmetic (+, -, *, /), comparison (==, !=, <, >, <=, >=), or logical (and , or).
UnaryOp	op: String expr: Expression	A unary operation applying a prefix operator (- or not) to a single expression.
LetStmt	name: String value: Expression	A variable declaration introducing a new binding in the current environment frame.
AssignStmt	name: String value: Expression	An assignment statement updating an existing variable binding in the environment chain.
PrintStmt	expr: Expression	Evaluates the contained expression and outputs its result to the console.
ExprStmt	expr: Expression	A standalone expression evaluated for its side effects (e.g., a function call acting as a statement), terminated by a semicolon.
IfExpr	cond: Expression then_branch: Block else_branch: Block	A conditional expression. Evaluates cond ; if true, evaluates then_branch , otherwise evaluates else_branch .

Node Type	Fields (Children)	Description
FunExpr	params: List of Strings body: Block	An anonymous function definition, capturing its parameter list and the block of code to execute.
CallExpr	func: Expression args: List of Expressions	A function invocation. Evaluates func to a closure, evaluates the arguments, and executes the closure body in a new environment frame.